Stacking

Stacking (Stacked Generalization) is an **ensemble machine-learning technique** where predictions from multiple different ML models are combined and given to another model, called a **meta-model**, which learns how to make the final prediction.

Simple Definition

> **Stacking combines multiple ML models by using their predictions as inputs to a final model that produces the final prediction.**

Components of Stacking

A. Base Models

These are the first-level models.

For example:

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost
- SVM
- Neural Network

They can be **different algorithms** or different configurations of the same algorithm.

B. Meta-Model

The second-level model receives the predictions from the base models.

Examples:

- Logistic Regression
- Linear Regression
- Random Forest
- XGBoost
- Neural Network

Why Use Stacking?

Different ML models have different strengths.

For example:

- **Logistic Regression** → good at simple linear relationships
- **Decision Trees** → good at rule-based relationships
- **Random Forest** → good at nonlinear relationships and robustness
- **XGBoost** → often strong on structured/tabular data
- **Neural Networks** → can learn complex patterns

Important: Avoid Data Leakage

This is one of the most important technical points in stacking.

You should **not simply train a base model on the entire training dataset and then use its predictions on that same data to train the meta-model**.

Why?

Because the base model has already seen those samples. Its predictions can therefore be unrealistically good, causing the meta-model to learn from **leaked information**.

Correct approach: Out-of-Fold Predictions

Technique	Main Idea
Bagging	Train models independently and aggregate their predictions
Boosting	Train models sequentially, focusing on previous errors
Voting	Combine predictions through majority vote/averaging
Stacking	Use another model to learn how to combine model predictions

```
Historical Sales ────────→ Random Forest ──────┐
Inventory Data ───────────→ XGBoost ───────────┤
Supplier Data ────────────→ Logistic Regression ──→ Meta Model
Lead Time ─────────────────→ SVM ──────────────────┤
Promotions ───────────────→ Neural Network ────┘
                                       ↓
                                  Stockout Risk
                                  = 87%
```

Example for Your Procurement Project

For an **FMCG Smart Inventory / Procurement system**, stacking could be useful for **demand or risk prediction**.

Advantages

Stacking can provide:

- Better predictive accuracy
- More robust predictions
- Ability to combine different algorithms
- Better handling of complex patterns
- Reduced dependence on one model

But these benefits are **not guaranteed**.

Disadvantages

Stacking also introduces complexity:

- More computational cost
- More models to maintain
- More difficult debugging
- Higher risk of data leakage
- More complex deployment
- Meta-model training requires careful validation
- Increased inference latency

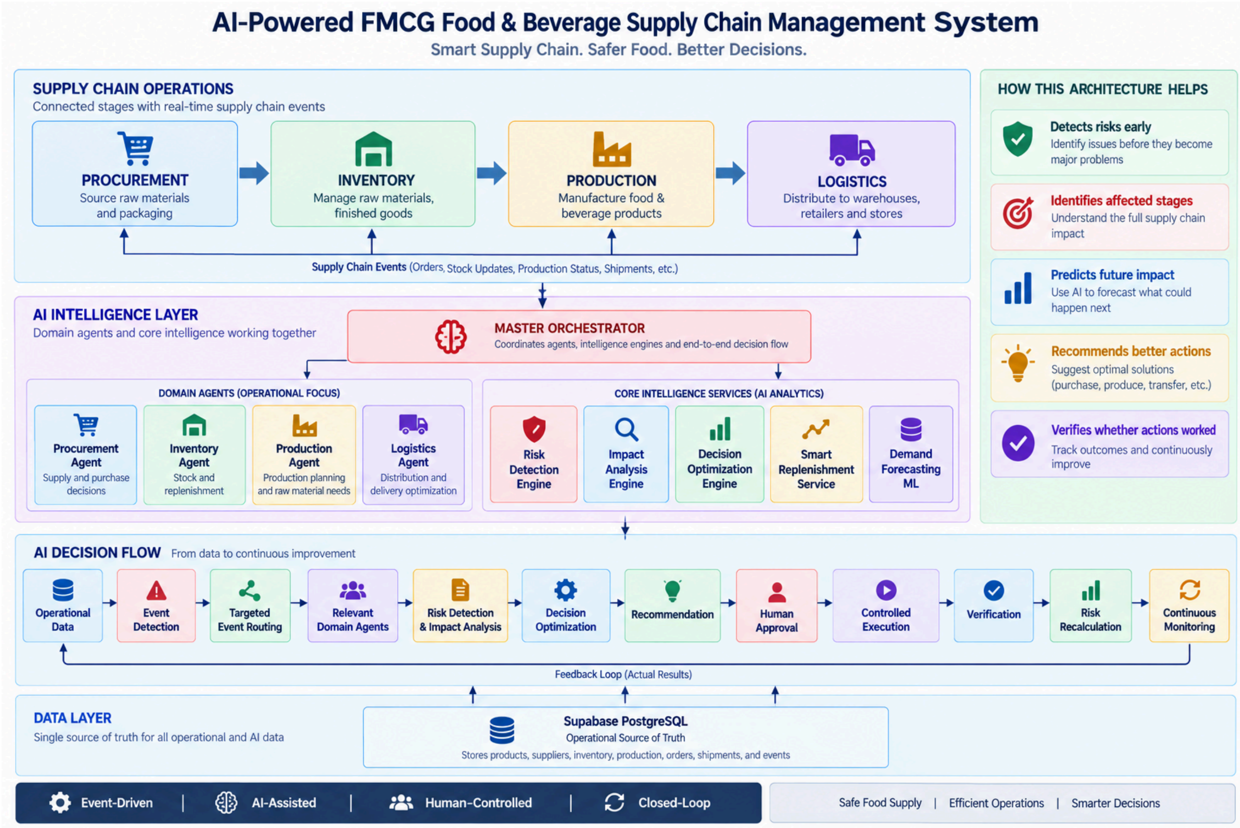
Stacking in One Sentence

> **Stacking is an ensemble learning method that combines predictions from multiple base models and uses a meta-model to learn the optimal final prediction.**

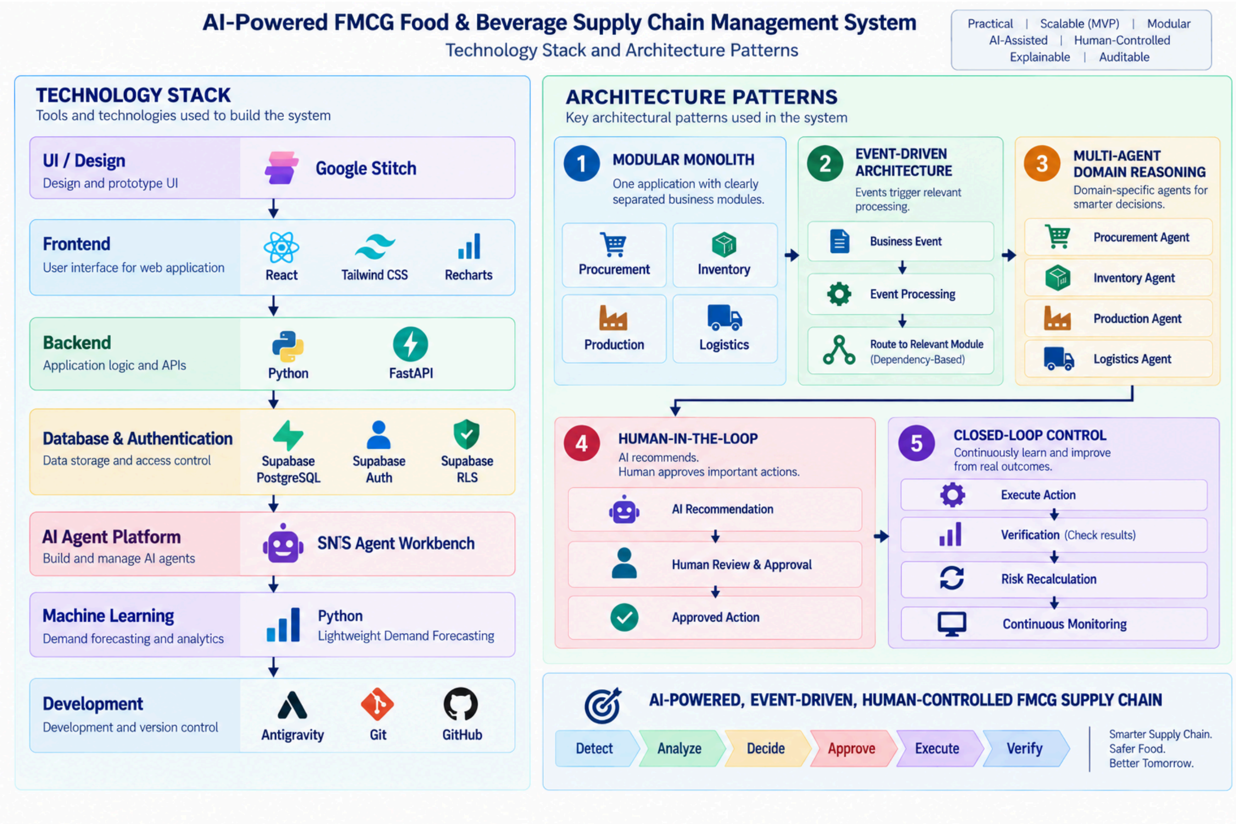
Easy memory trick

Base Models → Predictions → Meta-Model → Final Prediction

image


AI-Powered FMCG Food & Beverage Supply Chain Management System
Smart Supply Chain. Safer Food. Better Decisions.
SUPPLY CHAIN OPERATIONS
Connected stages with real-time supply chain events
PROCUREMENT
Source raw materials and packaging
INVENTORY
Manage raw materials, finished goods
PRODUCTION
Manufacture food & beverage products
LOGISTICS
Distribute to warehouses, retailers and stores
Supply Chain Events (Orders, Stock Updates, Production Status, Shipments, etc.)
HOW THIS ARCHITECTURE HELPS
Detects risks early
Identify issues before they become major problems
Identifies affected stages
Understand the full supply chain impact
Predicts future impact
Use AI to forecast what could happen next
Recommends better actions
Suggest optimal solutions (purchase, produce, transfer, etc.)
Verifies whether actions worked
Track outcomes and continuously improve
AI INTELLIGENCE LAYER
Domain agents and core intelligence working together
MASTER ORCHESTRATOR
Coordinates agents, intelligence engines and end-to-end decision flow
DOMAIN AGENTS (OPERATIONAL FOCUS)
Procurement Agent
Supply and purchase decisions
Inventory Agent
Stock and replenishment
Production Agent
Production planning and raw material needs
Logistics Agent
Distribution and delivery optimization
CORE INTELLIGENCE SERVICES (AI ANALYTICS)
Risk Detection Engine
Impact Analysis Engine
Decision Optimization Engine
Smart Replenishment Service
Demand Forecasting ML
AI DECISION FLOW
From data to continuous improvement
Operational Data
Event Detection
Targeted Event Routing
Relevant Domain Agents
Risk Detection & Impact Analysis
Decision Optimization
Recommendation
Human Approval
Controlled Execution
Verification
Risk Recalculation
Continuous Monitoring
Feedback Loop (Actual Results)
DATA LAYER
Single source of truth for all operational and AI data
Supabase PostgreSQL
Operational Source of Truth
Stores products, suppliers, inventory, production, orders, shipments, and events
Event-Driven | AI-Assisted | Human-Controlled | Closed-Loop
Safe Food Supply | Efficient Operations | Smarter Decisions

AI-Powered FMCG Food & Beverage Supply Chain Management System
Component Architecture



AI-Powered FMCG Food & Beverage Supply Chain Management System
Technology Stack and Architecture Patterns
Practical | Scalable (MVP) | Modular
AI-Assisted | Human-Controlled
Explainable | Auditable
TECHNOLOGY STACK
Tools and technologies used to build the system
UI / Design
Design and prototype UI
Google Stitch
Frontend
User interface for web application
React
Tailwind CSS
Recharts
Backend
Application logic and APIs
Python
FastAPI
Database & Authentication
Data storage and access control
Supabase PostgreSQL
Supabase Auth
Supabase RLS
AI Agent Platform
Build and manage AI agents
SN'S Agent Workbench
Machine Learning
Demand forecasting and analytics
Python
Lightweight Demand Forecasting
Development
Development and version control
Antigravity
Git
GitHub
ARCHITECTURE PATTERNS
Key architectural patterns used in the system
1 MODULAR MONOLITH
One application with clearly separated business modules.
Procurement
Inventory
Production
Logistics
2 EVENT-DRIVEN ARCHITECTURE
Events trigger relevant processing.
Business Event
Event Processing
Route to Relevant Module (Dependency-Based)
3 MULTI-AGENT DOMAIN REASONING
Domain-specific agents for smarter decisions.
Procurement Agent
Inventory Agent
Production Agent
Logistics Agent
4 HUMAN-IN-THE-LOOP
AI recommends.
Human approves important actions.
AI Recommendation
Human Review & Approval
Approved Action
5 CLOSED-LOOP CONTROL
Continuously learn and improve from real outcomes.
Execute Action
Verification (Check results)
Risk Recalculation
Continuous Monitoring
AI-POWERED, EVENT-DRIVEN, HUMAN-CONTROLLED FMCG SUPPLY CHAIN
Detect
Analyze
Decide
Approve
Execute
Verify
Smarter Supply Chain.
Safer Food.
Better Tomorrow.